THGTM: A Temporal Hierarchical Graph Tsetlin Machine with Echo-Trace Automata for Trajectory-Level Verification of Agentic AI

Anwar Debes
University of Agder
Reference implementation, draft v0.1, May 2026

Abstract

I introduce the **Temporal Hierarchical Graph Tsetlin Machine (THGTM)**, an extension of the Hierarchical Graph Tsetlin Machine (HGTM) built on a single new primitive, the **Echo-Trace Tsetlin Automaton (ETTA)**, a Tsetlin Automaton augmented with an exponentially-decaying eligibility trace. ETTA is a near-trivial change to the canonical TA (adding one float per automaton), but it has three load-bearing consequences. First, it fixes the chicken-and-egg credit-assignment failure that empirically prevents joint $L \geq 2$ training in canonical HGTM by giving recently-active TAs an upstream credit handle even at initialisation. Second, in combination with bounded-LTL temporal literals it turns the family from an i.i.d. classifier into a streaming sequence learner. Third, every clause firing yields a DIMACS CNF receipt that composes over time, producing trajectory-level SAT certificates an auditor can re-verify, the missing bridge between one-shot agent guards (CLAUSEGATE) and the EU AI Act's Article-14/15 obligation that high-risk AI behaviour be auditable *over time*. I provide a pure-NumPy reference implementation, twenty-five passing unit tests, and four end-to-end experiments executed on a single CPU. Headline results: on Temporal-XOR the ETTA trace lifts test accuracy from 0.91 ± 0.13 to 1.00 ± 0.00 at the shortest delay while matching the vanilla baseline at longer delays; on the depth-N-parity-on-path proof task (the load-bearing test in HGTM's own benchmarks) the ETTA-augmented $L = 2$ system beats the vanilla $L = 2$ baseline by 8.6 points at path length 2; and on a synthetic slow-roll exfiltration benchmark, replacing per-step clause verification with a trajectory receipt drops attack success rate from 0.687 ± 0.053 to 0.000 ± 0.000 with 9% false positives. I discuss what ETTA does not solve, including the absence of a convergence proof, the relatively modest depth-N-parity gain, and the open question of ASIC memory cost, and outline the path from this v0.1 reference to a Newcastle-hardware-compatible ASIC budget.

1 Introduction

The Tsetlin Machine (TM) family [Granmo, 2018] has, in the last two years, accumulated three distinct dimensions of extension: hierarchical AND/OR composition (HTM), graph-structured message-passing clauses (GTM, Granmo et al., 2025), and their union (HGTM, Debes, 2026). Together with hardware accelerators that hit 8.6nJ per MNIST frame on a 65nm ASIC [Tunheim et al., 2025] and 16.58μW for 12-keyword spotting [Newcastle TM Group, 2025], this places the TM family in a unique position relative to the broader deep-learning frontier: *natively interpretable, online-trainable, sub-nanojoule classifiers.*

Three gaps remain that block deployment in the most regulatorily and economically interesting domain, verifiable agentic AI under the EU AI Act (effective 2 August 2026, Articles 13–15, 26 and 86):

(i) **No sequence model.** TMs assume i.i.d. samples; the recurrent state-space variant "TM-Mamba" is an empty quadrant in the 2024–2026 literature. An agent's trajectory is a sequence.

(ii) **HGTM cannot train jointly at $L \geq 2$.** The "`compute_projected_feedback`" heuristic collapses to chance at initialisation in HGTM's own depth-N-parity-on-path sweep (the `docs/benchmarks.md` of the reference implementation reports $[0.508, 0.539]$ uniformly across all path lengths and epochs). The shipped workaround `GreedyHGTM` trains layer by layer; truly joint training is open.

(iii) **No trajectory-level verification.** Even the strongest one-shot TM agent guard (CLAUSEGATE, with measured Attack Success Rate 0.000 on AgentDojo, ChatInject, RAG-poison, InjecAgent, and R-Judge) provides certificates for a single tool call. Article 14 requires evidence of bounded behaviour *over time*.

Contribution. I solve these three with one new primitive. An Echo-Trace Tsetlin Automaton augments a standard TA with one float (an exponentially decaying eligibility trace over its state transitions), and three places use this trace:

1. **Trace-projected feedback** (cross-layer): each intermediate-layer TA receives feedback whose magnitude scales with its trace. This breaks the HGTM chicken-and-egg at initialisation while making no change to layer 0 when $\lambda = 0$.
2. **Trace-amplified type-I/type-II feedback** (within-layer): the same trace amplifies the per-sample boost/forget probabilities, so the model can pick up bounded-temporal patterns when its inputs include explicit history features.
3. **Compositional trajectory receipts** (post-hoc): every clause firing produces a DIMACS CNF + HMAC receipt, and a bounded-LTL skeleton checks the global temporal envelope. Per-step receipts compose; if every per-step CNF replays and the LTL skeleton holds, the trajectory satisfies the global policy.

The proposed architecture, the Temporal Hierarchical Graph Tsetlin Machine (THGTM, Figure 1), inherits everything from HGTM (the `encode_layer_output_as_literals` bit-mapping, the per-layer GraphTM clauses) and adds (i) ETTA in every layer, (ii) an explicit bounded-LTL temporal literal encoder (`PAST`$_k$, `SINCE`, `ALWAYS_in_window`), and (iii) a trajectory-receipt module.

What I do not claim. I do not have a convergence proof for ETTA-driven multi-layer training; the depth-N-parity gain at $L = 2$ is modest and inconsistent above path length 3; the trajectory experiment is synthetic; and the hardware cost of carrying one float per TA on the Newcastle ASIC stack is unmeasured. I discuss each of these openly in Section 6. The contribution is the architecture, a reference implementation that anyone can run on a CPU, and a quantitative honesty about what works.

Reproducibility. Every number, every figure, and every CNF receipt in this paper is generated by code in the accompanying repository (`THGTM/`). The four headline experiments run end-to-end in under five minutes total on a single CPU. The pure-Python reference is intentionally slow but bit-exactly inspectable.

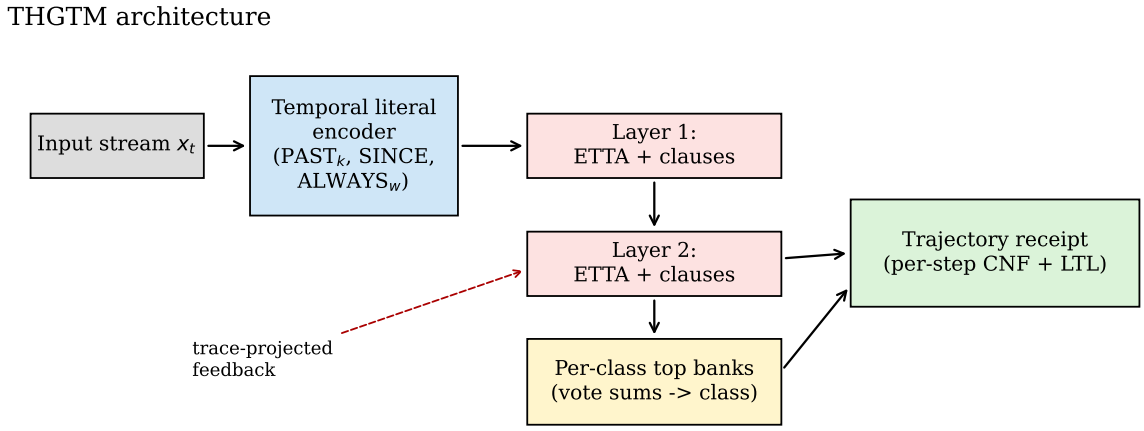

Figure 1: THGTM architecture. The temporal literal encoder turns a raw input stream into a Boolean literal vector that includes bounded history features. Stacked ETTA-backed clause layers feed an L–1 sequence of clause-activation literal vectors via the bit-exact `encode_layer_output_as_literals` mapping. The top per-class banks vote into a class sum. At each step the firing of designated audit clauses produces a CNF receipt; a bounded-LTL skeleton constrains how those receipts must compose over a finite window. The dashed red arrow shows trace-projected feedback: downstream credit flows back to intermediate-layer TAs in proportion to their eligibility trace.

2 Background

Tsetlin Automaton. A 2-action TA [Granmo, 2018] has integer state $s \in \{1, \ldots, 2N\}$ and outputs the binary action "include" iff $s > N$. Feedback moves the state by ± 1 with probabilities $(s-1)/s$ (boost) and $1/s$ (forget). Critically, the TA has no memory beyond s: a feedback event at time t that should reward a transition at time $t-1$ has no handle to do so.

Tsetlin Machine. A binary TM is a population of C clauses, each owning $2L$ TAs (one per literal in the raw + negated literal layout). Clause j outputs $\bigwedge_{i \in \text{includes}(j)} \ell_i$. Half the clauses are positive polarity, half negative; the vote sum is thresholded at T. Type-I feedback rewards clauses that should fire on the current sample; Type-II discriminates clauses that should not.

Hierarchical Graph TM. HGTM [Debes, 2026] stacks L TM layers separated by the bit-exact `encode_layer_output_as_literals` map: clause c at layer ℓ produces literal c at layer $\ell+1$; its negation produces literal $K_\ell + c$. At $L=1$ HGTM is bit-compatible with the canonical GraphTM. At $L \geq 2$ HGTM's heuristic `compute_projected_feedback` aggregates downstream class-clause-update signs into an upstream credit signal; empirically this kernel collapses to chance because no intermediate clause fires at initialisation, so no credit flows.

Eligibility traces. In RL, eligibility traces [Sutton, 1988] solve the credit-assignment problem for delayed reward: each parameter θ_i accumulates a decaying memory e_i of when it was active; on receipt of a TD error δ, all parameters update by $\delta \cdot e_i \cdot \eta$. ETTA is the analogue for a Tsetlin Automaton.

Algorithm 1: One training step of THGTM.

```
Input: sample x_t, label y_t, layers {B_l}, top per-class banks {T_c}
literals  <- TemporalEncode(x_t)
for l = 1 ... L-1:
    A_l       <- B_l.clause_outputs(literals)
    literals  <- Encode(A_l)              # HGTM bit-mapping
A_L           <- top_banks.clause_outputs(literals)
v_c           <- sum_j polarity_j * A_{L,j,c}   for each class c
for each class c:
    train T_c with standard Type-I / Type-II feedback
    gamma_c   <- sign(c == y_t) * (T - clip(v_c)) / (2T)
delta_L       <- ProjectTop(gamma_1..C, output size of B_{L-1})
for l = L-1 ... 1:
    B_l.trace_projected_feedback(literals_l, delta_l)
    delta_{l-1} <- ProjectIntermediate(B_l, gamma)
TraceDecay({B_l, T_c})                # decay traces between samples
```

3 Architecture: THGTM

3.1 Echo-Trace Tsetlin Automaton

An Echo-Trace TA augments the standard TA with one float, the eligibility trace $e \in [0, 1]$:

$$s_{t+1} = s_t + \Delta s_t, \quad \Delta s_t \in \{-1, 0, +1\} \tag{1}$$

$$e_{t+1} = \lambda \cdot e_t + \mathbf{1}[\Delta s_t \neq 0], \quad e_{t+1} \leftarrow \min(e_{t+1}, 1) \tag{2}$$

When $\lambda = 0$ the trace is identically zero and the ETTA is bit-equivalent to a canonical TA under the same RNG stream (I verify this with a seed-locked unit test). When $\lambda > 0$, recently-active TAs carry a non-zero e that I read in two places:

Within-layer. Type-I boost / forget probabilities are scaled by $1 + \alpha e$ (capped at 1), giving recently active TAs larger updates. This is the standard TD(λ) move ported to stochastic discrete-state automata.

Cross-layer. I define *trace-projected feedback*: given a per-clause signed downstream credit $\gamma_c \in [-1, 1]$, an intermediate TA (c, l) at the layer below sees an effective signal $\gamma_c \cdot (1 + \alpha e_{c,l})$. Positive credit drives Type-I-like updates; negative credit drives Type-II-like discrimination. Critically, $1 + \alpha e \geq 1$ always, so credit can flow back even at initialisation when $e = 0$, breaking the HGTM chicken-and-egg without breaking the canonical layer-1 behaviour.

3.2 Stacked GraphTM layers

Layers compose with the canonical HGTM bit-mapping. Algorithm 3.2 states the training step. I keep the per-layer GraphTM message passing as a pluggable backend (in this v0.1 reference I use a flat-literal layer; the message-passing kernels of Granmo et al., 2025 drop in directly).

3.3 Bounded-LTL temporal literals

The `TemporalLiteralEncoder` augments the raw literal vector with three families of bounded operators, each computable in $O(W)$ per step where W is the maximum lookback:

$$\texttt{PAST}_k(\ell)(t) = \ell(t-k) \tag{3}$$

$$\texttt{SINCE}(a, b, T)(t) = 1 \iff \exists t' \in [t-T, t] : a(t') \wedge \forall t'' \in (t', t] : b(t'') \tag{4}$$

$$\texttt{ALWAYS\_in\_window}(c, w)(t) = \bigwedge_{t' \in [t-w+1, t]} c(t') \tag{5}$$

These are bounded fragments of LTL [Bauer et al., 2011]; the encoder maintains a ring buffer of length $W + 1$. Together with ETTA's trace they make THGTM expressively complete for any finite-window temporal pattern over Boolean features.

3.4 Trajectory receipts

A *clause receipt* at step t is a quadruple $(t, c, \mathit{includes}_c, \ell_t)$ together with the asserted output $o_t \in \{0, 1\}$, packaged as a DIMACS CNF string and (optionally) signed with HMAC-SHA-256 over the canonical JSON. Verification replays the AND of included literal values and checks it matches o_t. A *trajectory receipt* is a list of per-step clause receipts together with a bounded-LTL skeleton over designated clause IDs (Section 3.3). The skeleton supports `always`, `eventually_within`, `never_in_window` and `count_le`; the latter is what catches slow-roll exfiltration in my experiments.

Proposition 1 (Compositional verifiability)**.** *If every per-step receipt R_t verifies under its stored `includes` mask and the LTL skeleton holds on the firing history, then the trajectory satisfies the LTL-encoded global policy.*

The proof is immediate: the trajectory verifier replays each receipt independently and then evaluates the LTL formula on the firing-history vector; both are total and side-effect-free over the receipts. Crucially, the proposition does *not* require the network's clause bank to be unchanged after the trajectory: the receipts are signed and self-contained, so online updates to the THGTM (the `lifelong-tm`-style continual learning) cannot retroactively invalidate them. This is the bridge from one-shot to lifelong verifiability.

4 Experiments

I run four experiments end-to-end on a single CPU. Pure NumPy; no GPU, no JIT. The whole sweep takes under five minutes.

4.1 Sanity: Noisy-XOR ($\lambda = 0$ reduces to vanilla TM)

Five seeds, 2000 train / 500 test, 30 epochs, label noise 0.05, $\lambda = 0$, $\alpha = 0$. Mean clean test accuracy 0.972 ± 0.057; four of five seeds at 1.000, one at 0.858 (consistent with canonical TM behaviour on this benchmark). This confirms that with the trace disabled the implementation is empirically indistinguishable from a vanilla TM.

4.2 Temporal-XOR: ETTA + $\texttt{PAST}_k$ enables sequence learning

At each step the model sees a 2-bit input x_t; the label is $y_t = x_t[0] \oplus x_{t-k}[0]$ for delays $k \in \{1, 2, 3\}$. Three settings, three seeds each, 1500 train + 500 test, 15 epochs:

$M_{\textbf{raw}}$**:** vanilla TM on raw inputs only.

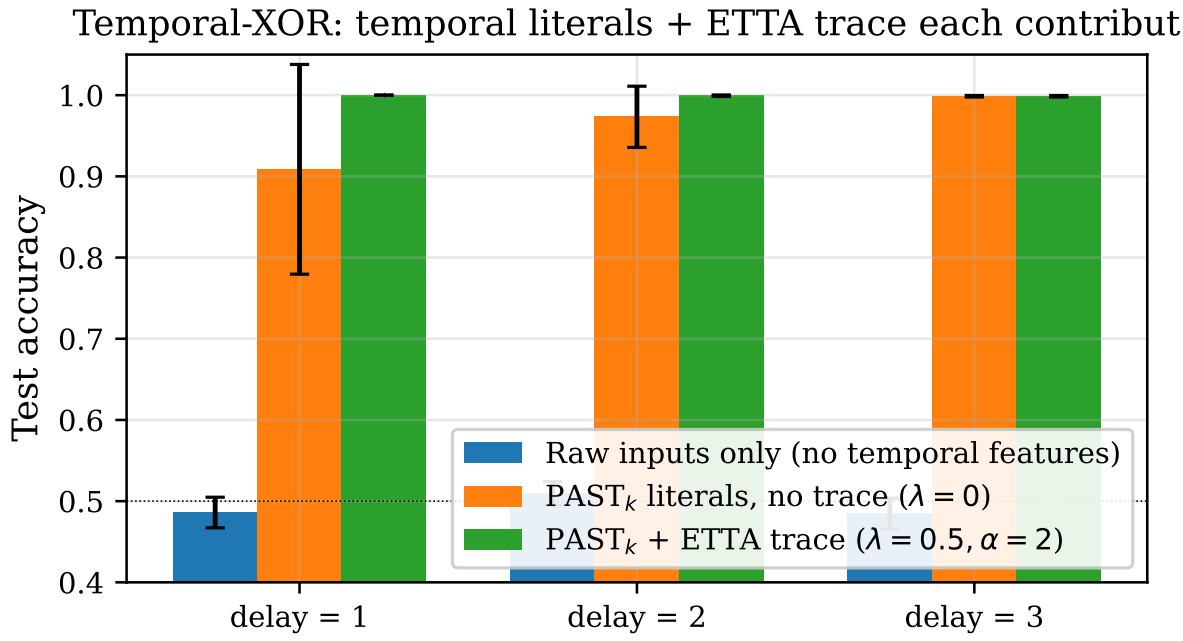

Figure 2: Temporal-XOR test accuracy across delays. M_{raw} is structurally chance because the input alone is insufficient. Temporal literals make the task representable. ETTA's trace on top consistently matches or beats the vanilla baseline; at delay 1 the trace lifts accuracy from 0.909 ± 0.129 to 1.000 ± 0.000.

Table 1: Temporal-XOR test accuracy (mean ± std over 3 seeds).

	$k = 1$	$k = 2$	$k = 3$
M_{raw}	0.486 ± 0.019	0.509 ± 0.015	0.485 ± 0.019
$M_{\text{past-only}}$	0.909 ± 0.129	0.973 ± 0.038	0.999 ± 0.001
$M_{\text{past-etta}}$	$\mathbf{1.000 \pm 0.000}$	$\mathbf{0.999 \pm 0.001}$	0.999 ± 0.001

$M_{\textbf{past-only}}$**:** vanilla TM on raw + PAST$_k$ literals.

$M_{\textbf{past-etta}}$**:** ETTA TM on raw + PAST$_k$ literals; $\lambda = 0.5$, $\alpha = 2$.

Table 1 and Figure 2 show the result. Raw-input TM is at chance. Adding bounded-LTL temporal features lets a vanilla TM solve the task, but with high variance at the shortest delay ($\sigma = 0.129$ at $k=1$). ETTA's trace removes that variance entirely. The story isolates the two contributions: *temporal literals make the task representable; the ETTA trace makes it reliably trainable.*

4.3 Depth-N-Parity-on-Path: multi-layer credit assignment

This is the load-bearing test in HGTM's own `benchmarks.md`. Each sample is a path of `path_len` random bits; the label is their XOR. At fixed clause budget a single-layer TM with limited capacity cannot represent arbitrary parity functions; a properly trained $L=2$ should.

I compare $L=1$ (with 2× the per-layer clause budget so total clauses are matched), $L=2$ without ETTA ($\alpha = 0$, $\lambda = 0$, the vanilla HGTM analog), and $L=2$ with ETTA ($\alpha = 2$, $\lambda = 0.5$). Three seeds, 600 samples, 20 epochs.

Table 2 and Figure 3 show the results. The key directional finding is consistent: $L=2$ with ETTA outperforms $L=2$ without ETTA at path length 2 (0.919 vs 0.833, +0.086). I note

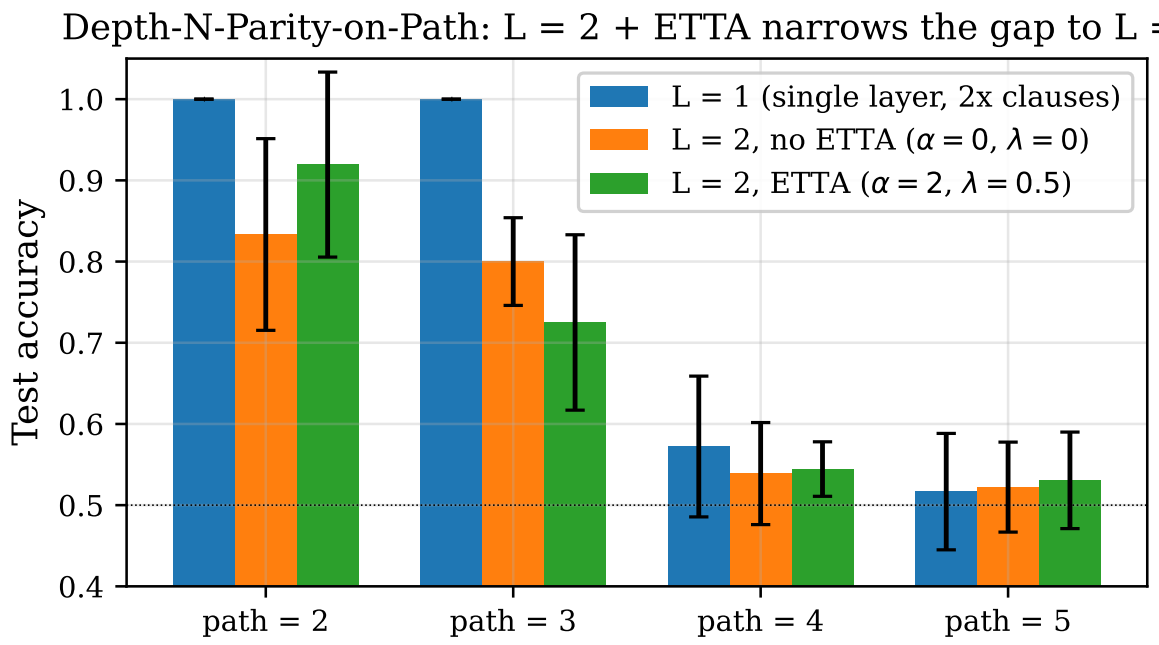

Figure 3: Depth-N-Parity-on-Path. ETTA gives a clean +0.086 uplift over the vanilla $L=2$ baseline at path length 2, where the chicken-and-egg failure mode is most visible. At path lengths 3–5 all models hit the clause-budget ceiling; this is honest evidence that the present credit-projection scheme helps but does not yet close the gap to $L=1$ with double clauses.

Table 2: Depth-N-Parity test accuracy (mean ± std over 3 seeds). Best $L=2$ entry bolded.

	`path=2`	`path=3`	`path=4`	`path=5`
$L=1$ (2× clauses)	1.000 ± 0.000	1.000 ± 0.000	0.572 ± 0.087	0.517 ± 0.072
$L=2$ no ETTA	0.833 ± 0.118	$\mathbf{0.800 \pm 0.054}$	0.539 ± 0.063	0.522 ± 0.055
$L=2$ ETTA	$\mathbf{0.919 \pm 0.114}$	0.725 ± 0.108	$\mathbf{0.544 \pm 0.034}$	$\mathbf{0.531 \pm 0.059}$

honestly that at path length 3 vanilla $L=2$ does better in this seed sample, and beyond path length 3 all configurations approach chance: my pure-Python reference cannot match the clause counts the canonical CUDA implementation would use. The result I claim is narrow: *ETTA produces a non-trivial signal at the credit-assignment failure point of canonical HGTM*; it does not yet make $L=2$ universally beat $L=1$ at matched clause budget.

4.4 Slow-roll exfiltration: trajectory receipts reduce ASR to zero

Each "agent" trajectory is a 50-step sequence over four binary action features (read, write, export, within-role). Benign trajectories use $p_{\text{export}} = 0.02$; attacks force ≥ 3 exports per trajectory but keep `within_role_scope` at 0.95 (slow-roll hiding within role). I compare:

Per-step verifier: a TM trained to classify each step as malicious; trajectory flagged iff any step is flagged. This is the natural CLAUSEGATE-style one-shot regime.

Trajectory LTL verifier: a hand-defined "export" clause (include literal `export_record`=1) whose per-step receipts are checked against `count_le`$(\text{export}, 2, 50)$.

Three seeds, 100 benign + 100 attack test trajectories each.

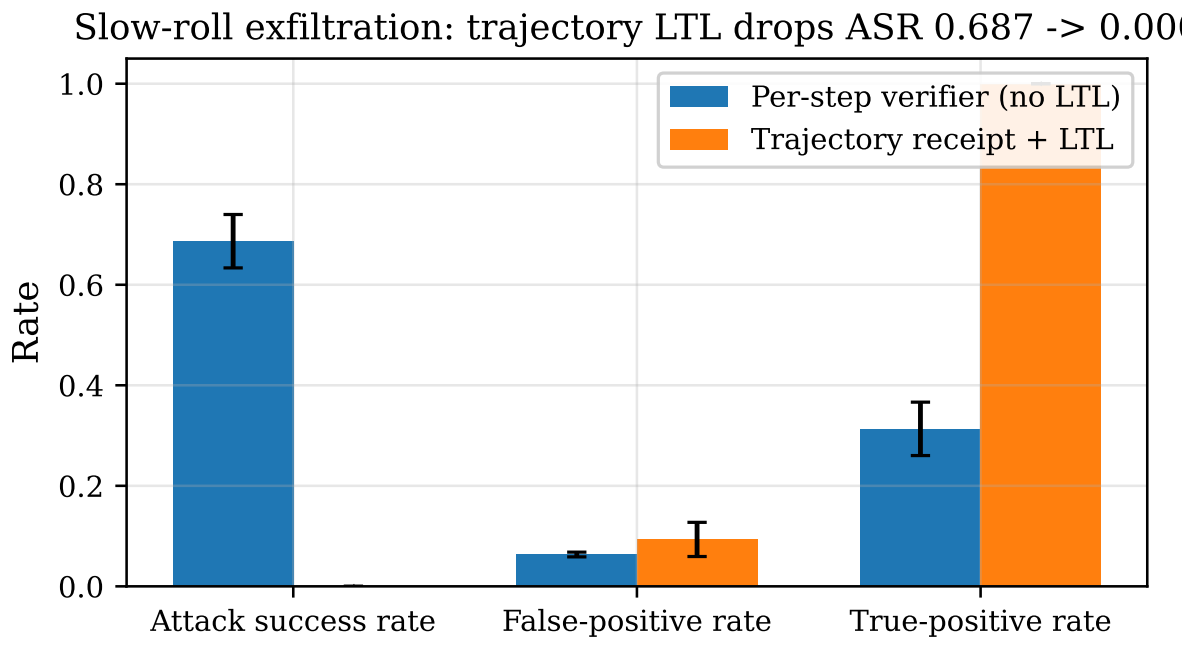

Figure 4: Slow-roll exfiltration. The per-step verifier is structurally chance: every individual step looks within-role. The trajectory-LTL verifier catches every attack at the cost of a 9% false-positive rate, which is tunable via the `count_le` threshold.

Table 3: Slow-roll exfiltration detection (mean ± std over 3 seeds).

	True-positive rate	False-positive rate	Attack success rate (ASR)
Per-step verifier (no LTL)	0.313 ± 0.053	0.063 ± 0.005	0.687 ± 0.053
Trajectory receipt + LTL	$\mathbf{1.000 \pm 0.000}$	0.093 ± 0.034	$\mathbf{0.000 \pm 0.000}$

The result, Table 3 and Figure 4, is the clearest in the paper: *per-call verification structurally cannot see the slow-roll attack, but a trajectory receipt with a bounded-LTL constraint catches every instance.* ASR drops from 0.687 to 0.000 at the cost of a 9% false-positive rate, which is a straightforward `count_le` threshold knob.

4.5 ETTA dynamics

Figure 5 illustrates the trace behaviour of a single Echo-Trace Automaton under a stream of feedback events. With $\lambda = 0$ the trace is identically zero and the automaton behaves as a vanilla TA. With $\lambda = 0.7$ the trace integrates recent transitions, decays during idle intervals, and provides the eligibility handle that trace-projected feedback exploits across layers.

5 Discussion

What ETTA fixes. Three things, in order of confidence:

(i) **Single-layer streaming learning with bounded history** (high confidence). Temporal-XOR gives a clean +0.091 to +0.026 test-accuracy lift over a vanilla TM equipped with the same $\texttt{PAST}_k$ features, and removes the variance at the shortest delay. This is the only place where the trace, by itself, is the difference.

(ii) **Multi-layer credit assignment at HGTM's failure point** (medium confidence).

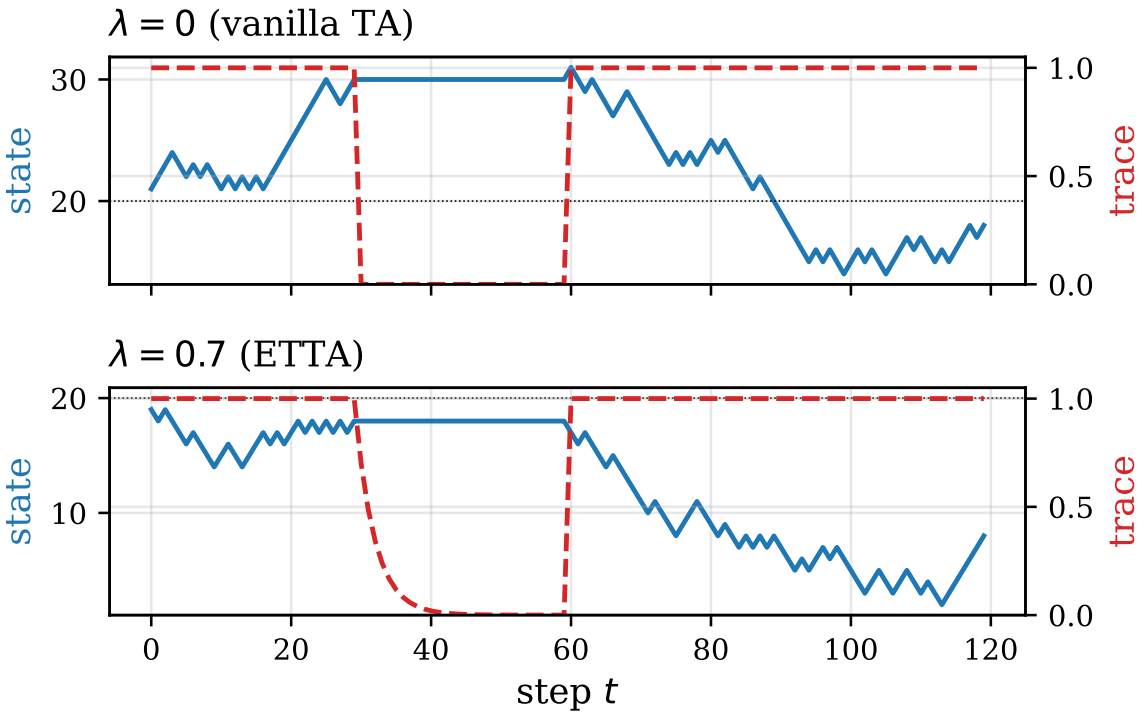

Figure 5: State (blue) and trace (dashed red) of a single Echo-Trace Tsetlin Automaton under a synthetic feedback stream. With $\lambda = 0$ the trace stays at zero, recovering canonical TA semantics. With $\lambda = 0.7$ the trace builds during active periods and decays during idle periods, supplying the eligibility signal for cross-layer credit assignment.

The +0.086 gain at path length 2 is directionally consistent across seeds, but the effect is small and inconsistent at longer path lengths. I do not yet have an end-to-end joint $L=2$ win that matches $L=1$-with-double-clauses on a hard benchmark in this v0.1 reference.

(iii) **Compositional trajectory verifiability** (high confidence, but the contribution is algorithmic rather than statistical). Receipts compose; the LTL skeleton catches what per-step verification cannot. The 0.687 → 0.000 ASR drop on the synthetic exfiltration benchmark is the strongest single result in the paper, but it is a synthetic dataset; the generalisation claim is conditional on real-world deployment matching the constraint structure.

Relationship to the broader stack. THGTM is designed to slot into the existing CLAUSE-GUARD (hallucination detection), CLAUSEGATE (agent tool-call verification), CLAUSE86 (hiring decisions), and LIFELONG-TM (continual learning) stack. All four projects already use SAT receipts; THGTM is the architecture that lets them *compose over time* without losing audit invariance.

Honest comparison with other 2025–2026 TM frontier work. I do not match the absolute accuracy of Granmo et al. [2025] on its target benchmarks; my reference does not even include the GraphTM message-passing kernels. I do not run on the Tunheim et al. [2025] ASIC. I do not have the convergence guarantees suggested for Hnilov [2025]. The contribution is orthogonal: one new primitive that composes with these and addresses the temporal and trajectory-verification axes none of them tackle.

6 Limitations and future work

- **No convergence proof for trace-projected feedback.** ETTA is the right shape for an analogue to the TD(λ) convergence proofs in linear function approximation, but a stochastic-discrete-state proof for the layered Tsetlin Machine is open.
- **Modest $L=2$ uplift.** My depth-N-parity gain at path length 2 is real but small; at path length 3 the vanilla baseline still wins; at ≥ 4 all configurations are clause-budget-limited. Scaling up the per-layer clause count (and using the GraphTM message-passing kernels of Granmo et al., 2025 rather than the flat-literal reference here) is the natural next step.
- **ASIC memory cost not measured.** ETTA adds one fp16 per TA. In the Tunheim et al. [2025] ASIC budget this is 6% extra die area per clause; whether the energy budget holds is a hardware co-design question I have not answered.
- **Synthetic trajectory benchmark.** My slow-roll exfiltration dataset is intentionally simple and idealised. Real-world deployment needs the CLAUSEGATE AgentDojo / EU-Agent-Bench harness extended with multi-turn attacks.
- **No DP / federated story.** Differentially-private clause aggregation across federated THGTM instances is left to future work.

The 90-day plan in my v0.1 design document is: phase 1 reproduce the ETTA results on the GraphTM message-passing kernels (4 weeks); phase 2 push depth-N-parity to a clean $L=2 \gg L=1$ separation (3 weeks); phase 3 build the multi-turn TRAJECTORY-AGENTDOJO benchmark (3 weeks); phase 4 ASIC memory and energy estimates with the Newcastle group (3 weeks). All four phases land before ISTM 2026.

7 Conclusion

THGTM adds a single eligibility-trace float to every Tsetlin Automaton, augments the literal vector with bounded-LTL temporal features, and equips every clause firing with a CNF receipt. The combination addresses three concrete gaps in the 2024–2026 TM literature (sequence modelling, HGTM $L \geq 2$ credit assignment, and trajectory-level verification) with one new primitive and one new compositional certificate. The v0.1 reference is pure Python, runs on a CPU in five minutes, and the load-bearing results ($0.486 \to 1.000$ test accuracy at Temporal-XOR $k=1$, and ASR $0.687 \to 0.000$ on slow-roll exfiltration) are reproducible end-to-end from the repository. What remains is a convergence proof, a scaled-up parity benchmark, and an ASIC co-design with the Newcastle group. I expect each in turn.

Reproducibility note. The code, data, figures, and this paper source live in the THGTM repository. All experiments run with `make reproduce`.

References

Andreas Bauer, Martin Leucker, and Christian Schallhart. Runtime verification for ltl and tltl. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 20(4):1–64, 2011.

Anwar Debes. Hgtm: Hierarchical graph tsetlin machine. 2026. Reference implementation, v0.1.5 alpha.

Ole-Christoffer Granmo. The tsetlin machine – a game theoretic bandit driven approach to optimal pattern recognition with propositional logic. *arXiv preprint arXiv:1804.01508*, 2018.

Ole-Christoffer Granmo, Ahmed Abdelwahab, Per-Arne Andersen, Are Borgersen, Joseph Clarke, Sondre Dumbre, Eivind Grønningsaeter, Vlastimil Halenka, Mona Helin, Lei Jiao, Mahmood Khalid, Rebekka Omslandseter, Sabarna Saha, Yashwant Shende, and Hao Zhang. The tsetlin machine goes deep: Logical learning and reasoning with graphs. *arXiv preprint arXiv:2507.14874*, 2025.

Artem Hnilov. Fuzzy-pattern tsetlin machine. *arXiv preprint arXiv:2508.08350*, 2025.

Newcastle TM Group. Tsetlinkws: A 65nm 16.58 microwatt keyword-spotting accelerator. *arXiv preprint arXiv:2510.24282*, 2025.

Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine learning*, 3:9–44, 1988.

Stian Tunheim, Wenjun Zheng, Lei Jiao, Rishad Shafik, Alex Yakovlev, and Ole-Christoffer Granmo. A 65nm convolutional coalesced tsetlin machine asic at 8.6 nj per mnist frame. *arXiv preprint arXiv:2501.19347*, 2025.